Project 1 – Abnormal EV Charging Sessions

Final Report: Data Preparation, Evaluation Setting, Models and Final Model

MSc in Informatics Engineering
Data Mining (MINDD)

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Students:
Amir Masnavi | 1221579@isep.ipp.pt
Ines Oliveira | 1231205@isep.ipp.pt
Manuela Leite | 1200720@isep.ipp.pt

Professor: Alberto Pinheira
Academic year: 2026/2027

Abstract

In this first task we studied 441,077 electric-vehicle charging sessions. The goal of the project is to predict, when a session starts, whether it will end abnormally. The file has no target column, so we built it from the end cause: the 13 causes that are faults count as abnormal, which gives 18.58% abnormal sessions. We checked this definition against the dataset's own history columns, and they match in about 100% of the rows.

Several columns are only known after the session ends (energy, costs, end and payment times, end cause), so we removed them from the predictors, together with the order creation time. We found no duplicates or invalid records. The missing values have a clear reason and the extreme values are real, so we did not remove any row. Volume, abnormal share and type of fault change over the two years, so we split the data by time and not at random.

We keep 29 features in 7 groups. The recent failure history of the charging post was the strongest signal we found.

In the second task we fixed how a solution will be evaluated, before training any model. A prediction is a risk alert raised when the session starts. The abnormal class is the minority and a missed failure costs more than a false alert, so the primary metric is the area under the precision–recall curve (PR-AUC), always read together with the abnormal share of the period. Models will be compared on four expanding-window folds, one per quarter, and the last quarter of 2021 stays untouched as the final test. Two baselines give the reference: a no-skill baseline, and the 7-day failure rate of the post, which already reaches a PR-AUC of 0.51 against 0.18.

In the third task we compared feature groups and trained individual and ensemble models. Charging-post history provides the largest improvement. User history adds a smaller benefit, mainly for returning users, while the tariff adds no measurable improvement beyond the hour. Gradient boosting on 27 features reaches a PR-AUC of 0.63, above the 7-day failure rate in every validation quarter. With alerts on 5% of the sessions, 88% of its alerts are real failures, but it still misses about three quarters of them.

In the fourth task we tuned gradient boosting, the random forest and the decision tree on the same folds. No tuned setting beat the Task 3 configuration by more than our tolerance, so gradient boosting stays the final model. We fixed its cut-off so that 10% of the sessions raise an alert, and only then scored the last quarter of 2021, which no earlier step had used. There it reaches a PR-AUC of 0.53, against 0.38 for the 7-day failure rate and 0.18 for no skill. Both scores are lower than on the validation quarters, which suggests that recent post failures predicted the next failure less well in that quarter.

The last chapter before the Synthesis brings the final choices together. One table compares the candidate, baseline and final models, another describes the final model, and a reproducibility statement explains how every result can be regenerated from the repository and the data file.

Contents

1 Introduction **1**
1.1 Target definition 1

2 Data Audit and Exploration **2**
2.1 Availability at prediction time 2
2.2 Data quality 2
2.3 What relates to abnormal termination 2
2.4 Changes over time 4

3 Preparation **4**
3.1 Chronological split 4
3.2 Features 5
3.3 Learned operations and class imbalance 6

4 Prediction and Evaluation Setting **6**
4.1 Prediction problem 6
4.2 Variables available at the start 7
4.3 Consequences of the two errors 8
4.4 Evaluation metrics 8
4.5 Temporal evaluation 8
4.6 Aggregation across folds 10
4.7 Baselines 10
4.8 Evaluation protocol 12

5 Individual and Ensemble Models **13**
5.1 Feature-group ablation 13
5.2 Individual models 14
5.3 Ensemble models 16
5.4 Comparison and alerts 17

6 Optimisation, Threshold and Final Evaluation **19**
6.1 Models chosen and search 19
6.2 Optimised against initial, and the final model 20
6.3 Decision threshold 20
6.4 Calibration 22
6.5 Final evaluation on the test period 22
6.6 What drives the final model 23

7 Final Model **24**
7.1 Candidate, baseline and final models 24
7.2 Final preprocessing, features, hyperparameters and threshold 24
7.3 Reproducibility statement 25

8 Synthesis **26**

Bibliography **28**

List of Figures

2.1 Univariate ROC-AUC on the training period, coloured by availability at the start . . . 2
2.2 Abnormal share by decile of six history variables (dashed line: training average) . . . 3
2.3 Monthly volume, abnormal share, new users and active posts . . . 4

4.1 Expanding-window folds, burn-in and final test period . . . 9
4.2 Baseline B1: precision–recall curves and PR-AUC per fold . . . 11

5.1 Feature-group ablation by fold . . . 13
5.2 Stacking without leakage . . . 17
5.3 All models by fold . . . 18

6.1 Effect of the cut-off . . . 21
6.2 Reliability of the final model . . . 22
6.3 Permutation importance of the final model . . . 23

Acronyms

AUC	Area Under the Curve
EV	Electric Vehicle
FN	False Negative
FP	False Positive
IQR	Interquartile Range
NaN	Not a Number (missing value)
PCA	Principal Component Analysis
PR-AUC	Area under the Precision–Recall curve
ROC-AUC	Area under the Receiver Operating Characteristic curve
SMOTE	Synthetic Minority Over-sampling Technique
SVM	Support Vector Machine
VIF	Variance Inflation Factor

1 Introduction

The project asks whether the information available when an Electric Vehicle (EV) charging session starts can predict that the session will end abnormally [1]. We work with an educational version of the China EV Charging Dataset [2]: 441,077 sessions and 47 columns, from 31 Dec 2019 to 31 Dec 2021, at 92 charging posts in 13 stations and 3 districts. All the numbers in this report come from our notebooks `Task1.ipynb`, `Task2.ipynb`, `Task3.ipynb` and `Task4.ipynb` (seed 42). Chapters 2 to 6 follow the four tasks, Chapter 7 gathers the final model and how to reproduce it, and Chapter 8 answers the project question.

We followed three rules during the whole project. First, a variable can only be a predictor if it is known when the session starts. Second, before fixing a problem in the data, we tried to understand it. Third, any value learned from the data (for example, the mean used for scaling) is learned from the training period only, which is how the data-preparation slides prevent leakage [3].

1.1 Target definition

The brief describes the target `is_Abnormal`, but the file does not include it. It only has `end_cause`, with 15 values. We decided that a session is *normal* when it ends with "Charging ends normally" or "User stops charging", because stopping is the user's own choice. The other 13 causes are equipment, communication or vehicle faults, so we label them *abnormal*.

To check this choice, we used the history columns of the dataset. For each session, they count how many earlier sessions of the same post or user were abnormal. We recounted them with our label and with an alternative where only "Charging ends normally" is normal. Our definition reproduces the supplied counts almost exactly and the alternative does not (Table 1.1), so the dataset was built with the same definition as ours.

Table 1.1: Candidate target definitions and agreement with the supplied history columns

Definition of "normal"	Post, 30 days	User, 30 days	Abnormal share
A: ends normally + user stops	100.00%	99.98%	18.58%
B: ends normally only	0.26%	35.85%	53.93%

With this definition there is about 1 abnormal session for every 4.4 normal ones. A model that always predicts "normal" would already be 81% accurate, so accuracy alone is not a good metric here [4]. We also noticed two details. The supplied rates are smoothed as $(\text{abnormal} + 1) / (\text{sessions} + 5)$, so a post or user with no history always gets the rate 0.2. In addition, 9,058 normal sessions (2.5%) delivered no energy, mostly users who stopped after about one minute. We kept them as normal, because removing them would require the energy, which is only known after the session.

2 Data Audit and Exploration

2.1 Availability at prediction time

Seven columns describe what happened during the session: energy, electricity cost, service charge, amount, actual payment, end time and payment time. `end_cause` is the outcome itself. Figure 2.1 shows why these columns are a problem. On its own, the energy, each money column and the duration separate normal and abnormal sessions with an Area under the Receiver Operating Characteristic curve (ROC-AUC) of 0.82–0.85, better than any variable known at the start (best 0.77). The payment delay is weaker (0.61), but it is also only known after the session. The reason is that abnormal sessions are short and deliver little energy, so these columns describe the result instead of predicting it. They are the "information leakers" that the data-preparation slides warn about [3], so all of them were excluded.

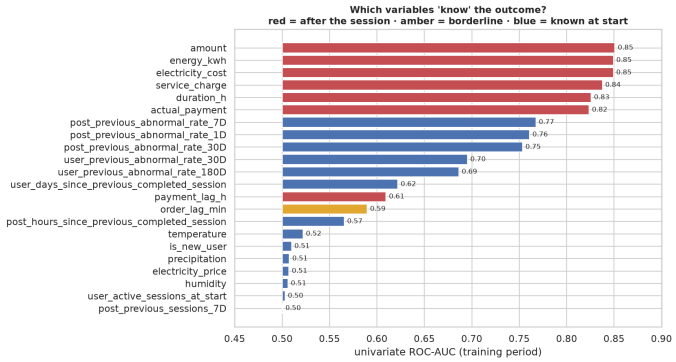


Figure 2.1: Univariate ROC-AUC on the training period, coloured by availability at the start

The order creation time was a harder case. It is stamped after the start in 56% of the sessions (57.2% in the training period, sometimes 16 h later), so we cannot be sure it exists when we make the prediction. Its link to the outcome is also weak (AUC 0.59), so we excluded it too. Weather has only one value per district and day. Following the brief [1], we treat it as a forecast that is known at the start.

2.2 Data quality

Table 2.1 lists the checks we made. In general, the records are in good condition. The missing values appear only when there was no earlier session or failure, so they are not really unknown. For the outliers, we first used the Interquartile Range (IQR) rule [3] and then looked at the extreme cases one by one. They turned out to be real situations, so we did not remove any row.

2.3 What relates to abnormal termination

We measured these relationships on the training period only, so that the validation and test periods do not influence our choices. Location matters, but mostly at the level of the post. The abnormal share is 14.6–24.1% per district and about 8–38% per station, while the posts range from 4.1% to 60.9%. The 10 worst posts produce 24.1% of the abnormal

Table 2.1: Data-quality checks, findings and decisions

Check	Finding	Decision
Formatting	Trailing tab in the order and payment times and in 18,370 location values	Stripped on load
Duplicates	No identical rows; 17 pairs where one account starts on two posts in the same second (fleet accounts)	Kept
Consistency	No negative values and no session ending before it starts; durations stop at 23.99 h; 54,621 sessions with zero energy	Kept (these are after-session fields)
Missing values	Only in 4 “time since previous” columns; every NaN matches its “no history” flag	Not deleted, not filled with the mean (Section 3.3)
Hidden codes	No −999 or “unknown” codes; values of 999 and above are real counts of fleet accounts	No action
Outliers (IQR)	Long right tails in counts and time-since columns (skewness 3–9)	Kept; `log1p` for linear models
Outliers (checked)	Posts idle for more than 14 days (39% abnormal on return), 21 accounts with more than 500 sessions, two storm days; the sessions flagged by Isolation Forest are 60% abnormal	Real and informative, so kept

sessions with only 10.2% of all sessions. Calendar and weather effects are small. Across the hours of the day the abnormal share moves between 14.7% and 20.5%, across the days of the week between 17.0% and 18.9%, and the weather effects are weak and not linear.

The history columns gave the clearest signal (Figure 2.2). When we split the 7-day post rate into deciles, the abnormal share goes from about 4% in the lowest decile to about 60% in the highest, so a post that failed recently tends to fail again. User history also helps, but less: its AUC is around 0.69–0.70, against 0.75–0.77 for post history.

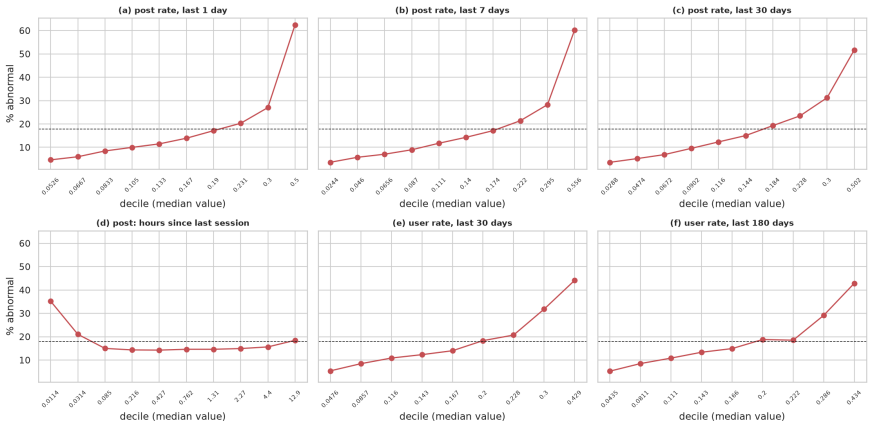


Figure 2.2: Abnormal share by decile of six history variables (dashed line: training average)

The users are very unequal. In the training period, 45% of the 34,294 accounts have only one session, but the top 1% of accounts have 34% of the sessions. New users (12% of the sessions) fail a little more often than returning users (20.2% against 17.5%).

2.4 Changes over time

Figure 2.3 shows that the data are not stable over the two years. Monthly volume drops to 1,391 sessions in February 2020, probably because of COVID-19, and reaches 25–28k sessions per month in autumn 2021. The abnormal share falls from 26.3% to 13.7% and later rises again to 22.9%. The type of fault also changes: connection faults add 7.1 percentage points to the abnormal share in Q1-2020, but only 1–2 points from Q3-2020 onwards. At the start of the file the history columns are still incomplete, which is why 31% of the sessions in January 2020 come from "new" users, against about 9–15% later. Finally, posts 17–20 stop on 28 Sep 2021. Some things are stable, though. The abnormal share of each post is similar in 2020 and 2021 (Spearman 0.81), which explains why post history works. Because of all the changes above, a random split would mix different periods and give results that are too optimistic, so we split the data by time, as is usual for data that are ordered in time [5].

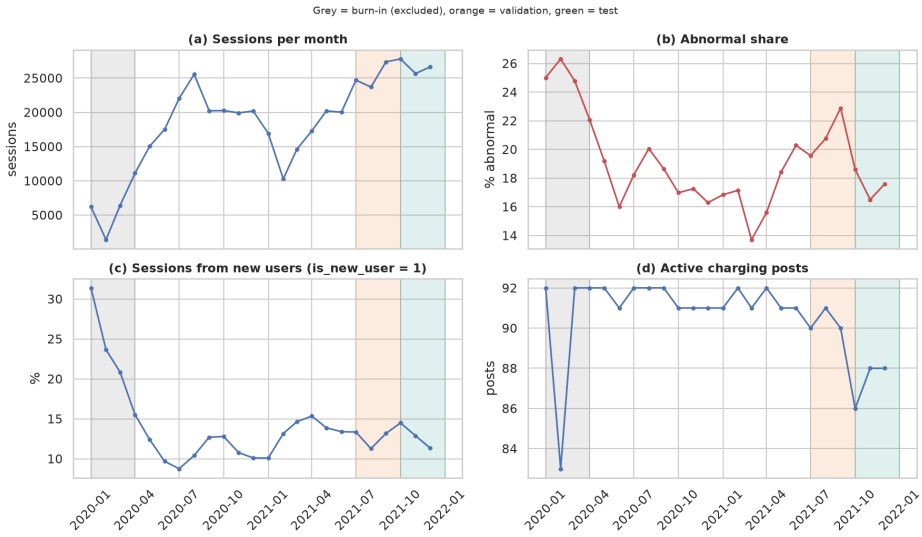


Figure 2.3: Monthly volume, abnormal share, new users and active posts

3 Preparation

3.1 Chronological split

We did not use the sessions before 1 April 2020 (the *burn-in*), because their history columns are incomplete and their fault mix no longer exists. The remaining data were split by start date into training, validation and test periods (Table 3.1). The label of a session is only known when it ends, so we also removed (*purged*) the 28 training sessions that end after the training cut-off. Otherwise their labels, which only exist after the cut-off, would leak into training [3].

Table 3.1: Chronological split

Period	Dates	Sessions	Abnormal
Burn-in (not used)	31 Dec 2019 to 31 Mar 2020	14,017	25.0%
Train	1 Apr 2020 to 30 Jun 2021	271,182	17.8%
Purged	Train starts that end after 1 Jul 2021	28	–
Validation	Jul to Sep 2021	75,769	21.1%
Test (used once)	Oct to Dec 2021	80,081	17.6%

The split shows two things that matter for the evaluation. First, 39.9% of the validation sessions and 61.3% of the test sessions come from accounts that never appear in training, so `user_id` cannot be a feature. Second, every post in validation and test was already seen in training, so we cannot test a brand-new post. The training and validation periods together form the development data. Chapter 4 divides them into four expanding-window folds, and the validation quarter of Table 3.1 is the fourth one, so we do not use a random K-fold.

3.2 Features

We keep 29 features in 7 groups: calendar (hour and day of week), tariff period, location (`post_id`, station and district), weather (temperature, humidity, precipitation and a new `rain_flag`), post history (8), user history (10) and concurrency (1). Table 3.2 lists the start-time columns we dropped.

Table 3.2: Start-time columns that were dropped

Column	Reason
`electricity_price`	Fixed by the tariff period
Six abnormal-count columns	Exact functions of the session counts and rates
Two post "no history" flags	Always 0 after the burn-in
`user_no_previous_completed_session`	Same as `is_new_user` in 99.9% of rows
`user_id`	Key field with 56k values; most test accounts are new

Some features are strongly related to each other. Only the user-history windows have a Variance Inflation Factor (VIF) above 10 (up to 65.4) [3], because the 180-day and 730-day windows contain almost the same sessions. Tree models and regularised linear models can deal with this, so they get all the features. For plain linear or distance-based models we built a smaller set. We went through the features from the best to the worst univariate AUC and kept a feature only if its $|\rho|$ with the features already kept was below 0.90. This leaves 19 of the 23 numeric features and brings the maximum VIF down to 8.6.

Principal Component Analysis (PCA) [3] was also tested. It needs 11 of the 23 components to keep 90% of the variance, and the first component hardly separates the classes (AUC 0.51). Since it compresses little and would mix post and user information, we did not use it. Discretisation [3] was only used to explore the data (deciles and weather bands), not as a model input.

3.3 Learned operations and class imbalance

Different models need different preparation, so we prepared two versions of the data. The tree version keeps the raw values, the NaN and the category codes. The linear version applies `log1p` to 8 skewed columns, fills the structural NaN with the training maximum (we read "never happened" as "a very long time ago") and keeps the flags. It then standardises the numeric columns, encodes hour and weekday with sine and cosine (the cyclical encoding recommended for circular variables), and one-hot encodes `post_id` and the tariff period [3]. The result has 126 columns and no NaN.

All operations that learn something are fitted on the training period only, inside a scikit-learn pipeline [6, 7] (Table 3.3). We checked what would happen if we fitted them on all periods instead, and the learned values change. For example, the fill value for "days since the user's last failure" would move from 543 to 702 days, which would leak information from the validation and test periods.

Table 3.3: Learned operations, all fitted on the training period

Operation	What is learned	Applies to
One-hot encoding / category codes	List of categories	Linear / tree version
Structural-NaN fill	Training maximum (flags kept)	Linear version
Standardisation	Mean and standard deviation	Linear version
Log-transform rule	Which columns are skewed	Linear version (8 columns)
Reduced feature set	Spearman ρ and AUC (training sample)	Plain linear / distance models
Class-imbalance treatment	Class weights or resampling	Not used: the models learn the natural prior (Chapter 5)
Outlier capping, PCA, rare-category grouping	Nothing	Not used / not needed

The classes (about $1:4.4$) are not balanced in the prepared data. Any class weights or resampling would be applied only inside the training folds, so that validation and test keep their real distribution. The models of Chapter 5 use neither and are judged with ranking and probability metrics. We avoided Synthetic Minority Over-sampling Technique (SMOTE) [3], because interpolating between sessions can create values that cannot exist, such as half a flag or a mix of two posts.

4 Prediction and Evaluation Setting

In the second task we fixed what exactly is predicted and how a solution will be judged, before training any model. The notebook `Task2.ipynb` fits no model. It builds the evaluation protocol and checks it with two simple baselines.

4.1 Prediction problem

The unit of observation is one charging session, that is, one transaction of one account at one charging post. The positive class is an abnormal ending (`is_Abnormal` $= 1$, one of the

13 fault causes). The negative class is a normal ending or a stop requested by the user, as defined in Section 1.1. In the development period (April 2020 to September 2021), 18.5% of the 346,979 sessions are abnormal.

The prediction is made once, at the start of the session, with only the information known at that moment. We designed it for preventive monitoring and maintenance prioritisation. The sessions that are starting, and through them the posts, are ranked by their risk of failure, so that an operator with limited capacity looks first at the riskiest ones. Predicting a session as abnormal is therefore a risk alert: this session, on this post, has an increased risk of ending in a failure. It is not a diagnosis and it does not say which fault will happen. The positive class also mixes 13 causes whose mix changes over time (Section 2.4), so a model could detect one type of failure well and miss another.

4.2 Variables available at the start

We placed each of the 47 supplied columns in one of the four categories of the brief [1] (Table 4.1). The notebook builds this table in code and checks that every column appears exactly once.

Table 4.1: Classification of the 47 supplied columns

Category	Columns	Which ones	Reason
Eligible predictors	27	Start time, post, station, district, weather (3), tariff period, post history (8), user history (10), concurrency	Known when the session starts; they give the 29 features of Section 3.2
Target construction only	1	`end_cause`	It is the outcome itself
Generated during or after the session	7	Energy, electricity cost, service charge, amount, actual payment, end time, payment time	They only exist once the session has ended (Section 2.1)
Excluded for another reason	12	`UserID`; order creation time; `electricity_price`, six abnormal counts, two post "no history" flags and `user_no_previous_completed_session`	Identifier; uncertain timing; ten redundant columns (Table 3.2)

We exclude the seven after-session columns and `end_cause` because a model that used them could not run when the session starts. `UserID` is excluded because 35% to 46% of the validation sessions of each fold come from accounts that are not in the training window. A model would only memorise accounts, and it would identify people. The user history is the legitimate substitute.

The eligible set depends on assumptions that the data cannot prove. Weather is taken as a forecast, as the brief says. The history columns are taken as supplied. In production they keep updating, so in a validation period they already contain the outcomes of earlier sessions of that period that had finished. User history needs the platform to identify the account at the start, otherwise the user group is missing (the cold-start case). Finally,

`post_id` is also an identifier, but the post is known at the start and is the most informative location level, so it stays here. The ablation of Section 5.1 shows that it adds nothing once the post history is known.

4.3 Consequences of the two errors

A False Positive (FP) flags a normal session as risky, and a False Negative (FN) lets an abnormal session pass unnoticed [4]. What they cost depends on the action that follows the alert. With automated monitoring, a false alert is one more line on a dashboard, while a missed failure means that the next sessions on that post are not watched. With an inspection or a maintenance visit, a false alert wastes technician time, while a missed failure leaves a faulty post in service. Failures cluster by post (Figure 2.2), so one missed failure is usually followed by more. Warning the driver is the most delicate action, because false warnings make drivers stop trusting the alerts, so we left it out of the main use.

For a single event, we consider the false negative the more expensive error, because the action after an alert is cheap compared with a failure that continues unseen. However, the number of false alerts is limited by the capacity of the operators, and a very long list of alerts becomes useless. The application therefore needs a good ranking, with the real failures near the top, and a fixed number of alerts that the team can handle.

We do not know the cost ratio $c = \text{cost(FN)}/\text{cost(FP)}$. With calibrated probabilities the best cut-off would be $t^* = 1/(1+c)$, which gives 0.50 for $c = 1$, 0.33 for $c = 2$ and 0.17 for $c = 5$. For c above 4.4 the cut-off is below the abnormal share and almost every session would be flagged. Fixing a cut-off now would be arbitrary. Instead, we chose metrics that judge the ranking and the quality of the probabilities, and we report the operational results for fixed alert budgets (5%, 10% and 20% of the sessions flagged). The cut-off will be chosen later, from the cost ratio and the capacity, as in the threshold exercise of the evaluation lab [8].

4.4 Evaluation metrics

Table 4.2 lists the metrics, which we chose before optimising any model.

We do not use accuracy, which gives the same importance to both classes [4]. Always predicting "normal" is right in 79–84% of the sessions of each fold and catches no failure. F1 and similar metrics need a cut-off that is not chosen yet, and F1 gives the same weight to both errors, so they can only be reported once the final cut-off exists.

4.5 Temporal evaluation

The sessions cannot be divided at random, because volume, abnormal share and fault mix change over time (Section 2.4) and a model is always used on future sessions. Training sessions must come before the sessions that evaluate them, as in the time-series cross-validation of forecasting practice [5]. We keep the burn-in and the test period of Section 3.1 and divide the development period into four expanding-window folds (Table 4.3 and Figure 4.1). Each fold trains on the sessions that started and ended before its validation quarter. The fourth fold is exactly the training and validation periods of Table 3.1.

The boundaries are calendar quarters, fixed before looking at any model result. A quarter is about one season. It is large enough for a stable estimate (every validation

Table 4.2: Evaluation metrics, their role and their limitation in this problem

Metric (role)	Why we use it	Limitation here
PR-AUC (primary)	The abnormal class is the minority and both errors matter. It measures how precise the alerts stay as more failures are caught, and it ignores true negatives [4, 9]	Its no-skill value is the abnormal share, which changes between periods, so we read it next to that share and as a lift
ROC-AUC (complementary)	The chance that an abnormal session is ranked above a normal one. It does not depend on the abnormal share, so it can be compared across periods	Can look optimistic with imbalanced classes [9] and says little about the top of the ranking
Brier score and log-loss (complementary)	They measure the quality of the probabilities [10, 6], which a cost-based cut-off needs	Only for models that output probabilities; both depend on the abnormal share
Precision, recall and correct, false and missed alerts per 1,000 sessions, at an alert budget (operational)	The budget is the practical limit and is the same for every model and period, and the counts turn the errors into workload	The budgets are assumptions, because the real capacity is unknown; the counts depend on the abnormal share of the period
Value per fold, standard deviation, worst fold, trend, folds won (stability)	A model should not be judged by its average only	Four folds give only a rough idea of the variability

Table 4.3: The four expanding-window folds (training always starts on 1 Apr 2020)

Fold	Training until	Validation quarter	Training sessions	Purged	Validation sessions	Abnormal share	Unseen accounts
1	30 Sep 2020	Oct to Dec 2020	111,470	37	60,364	16.8%	35.3%
2	31 Dec 2020	Jan to Mar 2021	171,832	39	41,797	15.8%	36.3%
3	31 Mar 2021	Apr to Jun 2021	213,641	27	57,542	18.2%	46.0%
4	30 Jun 2021	Jul to Sep 2021	271,182	28	75,769	21.1%	39.9%

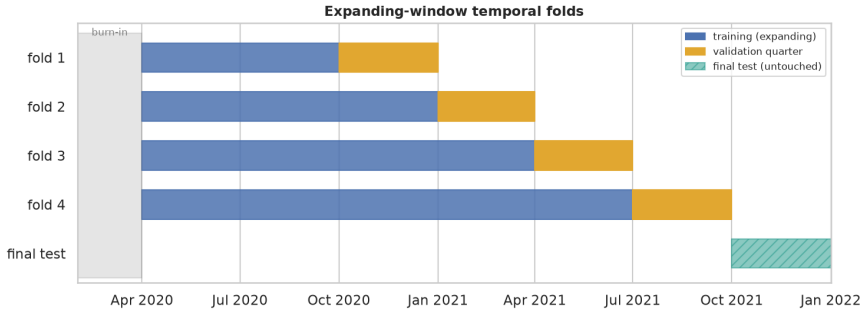


Figure 4.1: Expanding-window folds, burn-in and final test period

quarter has at least 6,600 abnormal sessions) and short enough to show changes over time. Four folds are a compromise: an earlier fold would train on less than six months of data, and monthly folds would give 12 small and noisy estimates. The window is expanding because the model that would be deployed is trained on all the past.

The folds are not equal. The validation quarters have between 41,797 and 75,769 sessions, and the training window grows from 6 to 15 months, so the first folds train on less data and do not include a full winter. We expect their scores to be a little pessimistic. The label of a session is only known when it ends, so the 27 to 39 training sessions per fold that were still running at the boundary are purged.

Every operation that learns from the data (Table 3.3) is fitted again inside each training fold, together with the model, and applied unchanged to the validation quarter. Class weights or resampling, if used, change the training rows only. Even a simple statistic differs between folds: the class prior goes from 17.7% to 18.9%, and no fold has the 18.5% of the whole development period. The difference is small here, but it would be larger for steps that use the target.

To protect the test period, we follow the golden rule of the evaluation slides: data used to evaluate a model cannot be used during its development [4]. The Task 2 notebook counts its 80,081 sessions and removes them right after reading the file. Model, hyperparameters and cut-off will be chosen with the four folds only. The cut-off will be chosen on the out-of-fold predictions of the four folds together and then frozen, as in the evaluation lab of the course [8]. The final model will be refitted on the whole development period and scored once on the test.

4.6 Aggregation across folds

We rank the models by the unweighted mean of the Area under the Precision–Recall curve (PR-AUC) of the four folds. We want a model that works in a future period, so each period counts once. Next to it we report the mean weighted by the number of validation sessions, which reflects the operational volume, and the pooled value over all out-of-fold predictions, which is needed to choose the cut-off but is dominated by the large folds. Stability is described by the value in each fold, the standard deviation, the worst fold and the trend. An improving trend is partly expected, because later folds train on more months, while a falling trend would be a sign of drift.

The abnormal share differs between folds, so the scores of two folds cannot be compared directly, but the difference between two models on the same fold can. We therefore compare models fold by fold, with the paired differences and the number of folds won. The evaluation slides compare models with a Wilcoxon test on the folds [4], but with four folds the smallest possible p-value of an exact Wilcoxon test is 0.125, even when a model wins in every fold, so we do not claim statistical significance for these comparisons. When two models are within about one standard deviation of each other, we will prefer the simpler and more stable one.

4.7 Baselines

We defined two baselines before training any model and evaluated them with the same folds and metrics (Table 4.4). B0 is the no-skill baseline: random scores for the ranking metrics and the training prior as a constant probability. B1 uses one supplied column as

its score, the smoothed abnormal rate of the post in the previous 7 days, and fits nothing. This column was the strongest single signal in Task 1 (Section 2.3).

Table 4.4: Baselines on the four validation quarters

Fold	Abnormal share	PR-AUC B0	PR-AUC B1	B1 lift	B1 ROC-AUC	Brier score B0	Brier score B1
1	16.8%	0.172	0.499	2.96	0.764	0.140	0.113
2	15.8%	0.162	0.456	2.88	0.749	0.134	0.111
3	18.2%	0.181	0.547	3.00	0.789	0.149	0.115
4	21.1%	0.210	0.540	2.56	0.788	0.168	0.132
Unweighted mean	–	0.181	0.511	2.85	0.772	–	–

B0 behaves as expected. Its PR-AUC is close to the abnormal share of each fold, so its lift is about 1, and its ROC-AUC is about 0.50. This means that "no skill" is not one number: it moves from 0.16 to 0.21 with the abnormal share. Fold 4 shows why this matters. Its PR-AUC is almost the same as in fold 3 (0.540 against 0.547), but its lift is clearly lower (2.56 against 3.00), because the no-skill level rose to 21%. For this reason we never read a PR-AUC without the abnormal share of its period.

B1 is already a strong reference. Its mean PR-AUC is 0.511 (weighted 0.516, pooled 0.524), with a standard deviation of 0.042 across folds. Its worst fold is fold 2 (0.456), the quiet first quarter of 2021. It beats B0 in the four folds, by 0.33 on average, and as a probability it has a lower Brier score than the constant prior in every fold. Figure 4.2 shows its curves. In folds 1 to 3 the precision stays above 0.8 up to a recall of about 0.2. Fold 4 is less precise at the top of the ranking, but it has the highest curve beyond a recall of about 0.4.

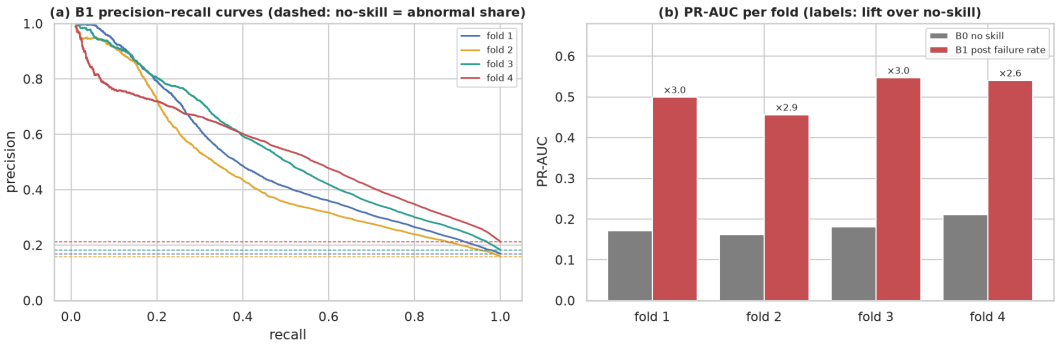


Figure 4.2: Baseline B1: (a) precision–recall curves per fold, with the no-skill level dashed; (b) PR-AUC per fold, with the lift over no skill

Table 4.5 translates these results into alerts. With a budget of 5% of the sessions, 74% of the B1 alerts are real failures (68–79% depending on the fold), but only 21% of the failures are caught. With a budget of 20%, half of the failures are caught and there are more false alerts than correct ones. B0 gives the chance level: its precision is the abnormal share (about 18%) and its recall is the budget. A model is worth its complexity only if it catches more failures than B1 at the same budget, without making the probabilities worse.

Table 4.5: Operational results of baseline B1 (mean of the four folds; alerts per 1,000 sessions)

Alert budget	Precision	Recall	Correct alerts	False alerts	Missed failures
5%	0.738	0.207	36.9	13.1	143.2
10%	0.594	0.330	59.4	40.6	120.7
20%	0.451	0.499	90.2	109.8	89.9

4.8 Evaluation protocol

Table 4.6 summarises the protocol. The notebook also saves it as `prepared/task2_protocol.json`, so the models of Chapter 5 use the same features, folds and metrics.

Table 4.6: Evaluation protocol, fixed before hyperparameter optimisation and model selection

Element	Decision
Admissible predictors	29 features in 7 groups, built from 27 original columns
Excluded variables	`end_cause` (target only); 7 columns generated during or after the session; `UserID` (identifier); order creation time (uncertain timing); 10 redundant columns
Primary metric	PR-AUC, read with the abnormal share of the fold and as a lift
Complementary metrics	ROC-AUC, Brier score and log-loss
Operational measures	Precision, recall and correct, false and missed alerts per 1,000 sessions, at alert budgets of 5%, 10% and 20%
Baselines	B0 no skill and B1, the 7-day failure rate of the post
Temporal strategy	Burn-in before 1 Apr 2020; four expanding-window folds with a purge (validation quarters from Q4-2020 to Q3-2021); test from 1 Oct 2021 untouched
Preprocessing	Fitted inside each training fold and applied unchanged to its validation quarter
Aggregation and comparison	Models ranked by the unweighted mean of the fold PR-AUC; weighted mean and pooled value also reported; standard deviation, worst fold and trend for stability; paired comparison fold by fold
Threshold and final test	Cut-off chosen on the pooled out-of-fold predictions and then frozen; one evaluation on the test

5 Individual and Ensemble Models

In the third task we trained the first models [1]. Every experiment follows the protocol of Chapter 4: the same four folds, metrics and baselines, with preprocessing fitted inside the training window of each fold. The rows of the test period are dropped right after the file is read, before the target is built, and no cut-off is chosen yet.

5.1 Feature-group ablation

We compared the four nested feature sets of the brief with one fixed reference model: gradient boosting with depth 6, learning rate 0.1, 200 iterations and at least 100 sessions per leaf, which we did not tune. With the same untuned model for every set, the differences come from the information and not from tuning, and the model handles categories, missing values and interactions without extra preparation [6]. We judge each step by its paired difference in PR-AUC, fold by fold, and treat differences below 0.005 as practically negligible. A bootstrap of each validation quarter (100 resamples of the sessions, the resampling of the evaluation slides [4]) gives a standard error of 0.004 to 0.006 for one model's PR-AUC. This gives a rough scale for the tolerance, not a test of a paired difference. Sessions share posts, users and periods, so the independent-session resampling can understate uncertainty.

Table 5.1: Feature-group ablation with the reference model (mean of the four folds; the selected set is S4 without `post_id` and the tariff; folds: folds with a gain)

					Gain over previous set	
Feature set	Features	PR-AUC	Lift	ROC-AUC	PR-AUC	Folds
S1 context and weather	9	0.290	1.60	0.660	–	–
S2 + tariff	10	0.290	1.60	0.659	+0.0006	3 of 4
S3 + post history	18	0.584	3.26	0.805	+0.294	4 of 4
S4 + user history	29	0.615	3.44	0.833	+0.031	4 of 4
Selected set	27	0.626	3.50	0.836	–	–
Selected, no user history	16	0.599	3.34	0.811	–	–

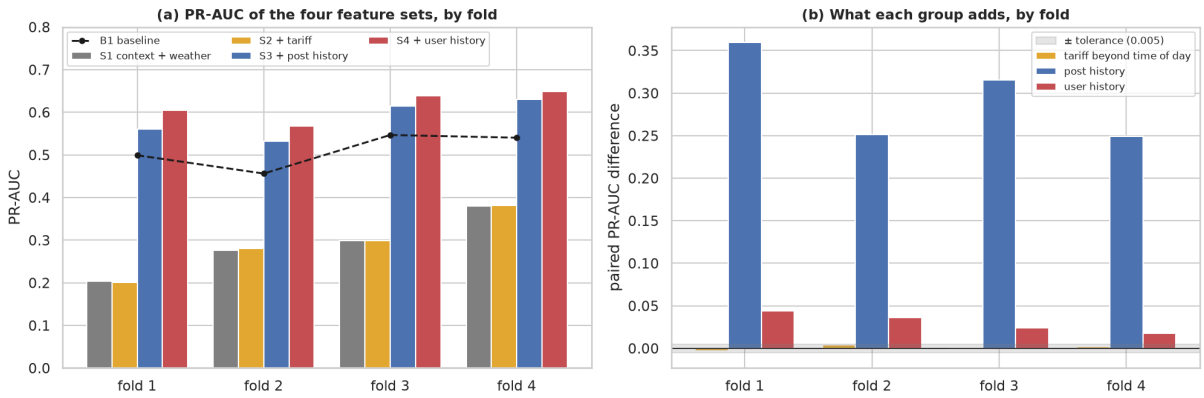


Figure 5.1: Feature-group ablation: (a) PR-AUC of the four sets by fold, with baseline B1 dashed; (b) paired gain of each added group by fold, with the tolerance band

Table 5.1 and Figure 5.1 answer the questions of the brief. The tariff gives no measurable improvement beyond the time of day in these experiments. Adding it changes the mean PR-AUC by +0.0006, and its fold differences (−0.0033 to +0.0038) stay inside the practical tolerance. The tariff period is a deterministic function of the start hour, so a model that already has the hour can rebuild it. Charging-post history provides by far the largest improvement. It raises the mean PR-AUC from 0.290 to 0.584, with a large gain in every quarter (+0.249 to +0.360). With it, the model is above B1 in every fold, by +0.061 to +0.090, so it does more than repeat the 7-day failure rate. User history adds a smaller benefit, +0.031, which is positive in every fold but falls from +0.044 in fold 1 to +0.018 in fold 4. The ordering of the sets is the same in every fold, but the size of the gains varies: the post-history gain differs by 0.11 between quarters and the user-history gain shrinks.

The benefit of user history is much larger for returning users: pooled PR-AUC rises from 0.609 to 0.640 (+0.032). About 13% of the validation sessions come from new accounts. For them the change is smaller and mixed: PR-AUC rises from 0.513 to 0.519 (+0.0057), just above our practical tolerance, while ROC-AUC falls from 0.730 to 0.727. A new account has no completed history, but its indicators and concurrency variables can still carry information, and fitting a model with another feature group changes predictions for all sessions. These results do not establish that user history cannot help a new account.

We then removed two variables from S4. Without `post_id` the mean PR-AUC rises by +0.0107, in all four folds. In these folds its identity adds no predictive benefit once the history is known. Removing it also reduces the opportunity to memorise devices whose failure patterns change over time. Removing the tariff as well changes the result by +0.0003, inside the tolerance, so it goes too. The selected set has 27 features and a mean PR-AUC of 0.626. Without the 11 user variables the mean result falls by 0.027 to 0.599, with a loss in every fold (−0.017 to −0.037), so by our rule user history stays. We add no new feature. The start-time columns were chosen in Task 1, post history provides the largest measured improvement, and a new history feature would need its own check that every session it counts had ended before the start.

Some groups raise operational or ethical concerns, the kind of risk that the introductory slides list for data mining [12]. The post history needs a live record of finished sessions, which always lags, because a label only exists when a session ends, up to 24 h later. If alerts lead to maintenance, failures may fall and the history may change, so the model can also reflect the maintenance policy. Station and district identify infrastructure rather than an account, although linkage with other records can still raise privacy concerns. The abnormal share goes from about 8% to 38% between stations, so the distribution of alerts should be monitored per station. User history is a behaviour profile tied to a persistent account. Its use therefore needs a clear purpose and data-minimisation safeguards [13]. A driver who used a faulty post could receive a higher score, so the user could be blamed for a failure of the infrastructure. We keep it because it improves the ranking, but only for an internal ranking of sessions and never for a decision about a driver. The set without user history (0.599) is the fallback.

5.2 Individual models

The brief lists five modelling principles and asks for at least three [1]. We chose a linear, a probabilistic and a tree-based classifier from the families of the predictive-models slides [14] (Table 5.2). The 27 features mix two categories, two circular calendar variables, weather, skewed counts and times, bounded rates and four variables with structural missing values.

About 18.5% of the sessions are abnormal. In Task 1 we decided not to resample, so every model learns the natural class prior.

Table 5.2: Individual classifiers: why they fit this problem and what they assume

Classifier	Why it fits	Assumption and risk	Interpretability and cost
Logistic regression (linear, L2 penalty)	The failure rate rises smoothly with the recent failure rate of the post, which a linear log-odds can follow; it outputs probabilities	Linear in the inputs, so skewed variables are logged and scaled; correlated windows make single coefficients unstable, which the penalty controls	One coefficient per input; seconds
Naive Bayes (categorical, Laplace estimate)	Few parameters, a posterior probability and a different inductive bias; it handles mixed types once numbers are discretised	Inputs independent given the class, which the overlapping 1-, 7- and 30-day windows break, so we expect overconfident probabilities	One likelihood table per input; seconds
Decision tree (CART)	Successive thresholds on rates and gaps, and their interactions, match the shapes seen in Task 1; no scaling needed	Unstable, since small changes in the data give another tree, and it overfits, so depth and leaf size are tuned	Readable rules; seconds

We left out two principles. k-nearest neighbours keeps every training session and needs a distance on inputs of the same scale, which is hard with counts, rates and categories together, and each of the 42,000 to 76,000 validation sessions of a fold would be compared with up to 271,000 training sessions. A kernel SVM needs a time between quadratic and cubic in the number of training sessions and returns a distance from the hyperplane rather than a probability [4, 14]. Both would need a subsample, which would throw away the history that carries the signal.

Each model gets the preparation that the slides recommend for it [14]. The logistic regression uses the linear version of Task 1: training maximum for the structural NaN, `log1p` for a fixed list of non-negative counts, gaps and precipitation, standardisation, one-hot station and district, and sine and cosine of hour and weekday, the cyclical encoding of the data-preparation slides [3]. The log-transform list is fixed from the meaning of the variables, rather than selected again using later periods. Naive Bayes keeps all eligible numeric inputs and cuts continuous ones into 10 equal-frequency bins learned on each training fold [3, 6]. Its category encoder reserves code zero for unseen categories, and Laplace smoothing uses $\alpha = 1$. We do not reuse the target-based reduced feature list learned in Task 1. The trees use the category codes of Task 1. The slides advise against imposing an arbitrary order on unordered categories, and we keep the codes for a reason: after `post_id` is dropped only 13 stations and 3 districts remain, and a tree can isolate any single code with two splits, so the order blocks no partition that it needs. Gradient boosting reads the same codes as categories, so for it no order is imposed at all.

The first version of our notebook also capped the logged columns and the number of active sessions at the 0.1% and 99.9% training quantiles, which contradicts the Task 1 decision not to cap outliers. We tested both. With the caps the mean PR-AUC of the

logistic regression changes by -0.0001 (0.5925 against 0.5924), and no fold moves by more than 0.0002. With 111,000 to 271,000 training sessions per fold, the extreme 0.1% are only a few hundred sessions. Their effect on predictive performance is negligible in this comparison. We kept the Task 1 preparation.

We tuned only the setting that controls the complexity of each model and kept the simplest configuration within the tolerance of the best one. For logistic regression, $C = 0.01$ and $C = 1$ give similar mean PR-AUC (0.5925 and 0.5923), with small differences in every fold, so we kept the stronger penalty. For the tree, depths 4 and 6 underfit (0.540 and 0.582), and depths 8 and 12 are tied (0.597 and 0.599), with a worse log-loss at depth 12 (0.354 against 0.347), so we kept depth 8. For Naive Bayes we keep 10 bins and Laplace smoothing fixed. It ranks sessions reasonably (ROC-AUC 0.814) but its PR-AUC is only 0.525, and its probabilities are worse than the constant prior (Brier score 0.172 against 0.148, log-loss 0.839 against 0.472). This is consistent with overconfidence when the conditional-independence assumption [14] fails: the overlapping windows describe some of the same failures. The tree and the logistic regression reach almost the same result (0.597 and 0.593), both 0.08 to 0.09 above B1, although one is linear and the other is not. This suggests that the available information strongly limits both models.

5.3 Ensemble models

We chose one ensemble for each of three ideas: lowering the variance of a model (bagging), lowering its bias (boosting) and combining models with different inductive biases (stacking). The predictive-models slides do not cover ensembles [14], so we explain each one with ideas that they do cover.

- The random forest [15] averages 200 trees, each fitted on a bootstrap sample [4] and with a random subset of features at each split. A single tree is unstable [14], and averaging trees fitted on different samples cancels much of that noise while keeping the low bias of a deep tree.
- Gradient boosting [16, 6] adds shallow trees one after another, each fitted to the remaining errors of the previous ones. A shallow tree can underfit, and the sequence can lower that bias step by step. We explicitly disable internal early stopping and compare fixed iteration counts, so that no random internal validation split is used.
- The stacking [17] combines the logistic regression, Naive Bayes and the tree with a logistic regression on their predictions. The three make different errors, and the meta-model learns how much to trust each. We did not use equal-weight voting, because one constituent is much weaker; stacking can learn a different contribution for each.

The random forest keeps the smaller leaf size (50 sessions, 0.622 against 0.616 with 200, better in all four folds). Gradient boosting keeps depth 6 with learning rate 0.1 and 200 iterations (0.626). Depth 3 is lower (0.621), and the slower setting with 300 iterations is tied with it.

The brief warns that the predictions that train the meta-model of a stacking must not leak [1], and that a random inner cross-validation lets later sessions shape the predictions for earlier ones. We built them in time order (Figure 5.2). The training window of each fold is cut into quarters. For every quarter after the first, the base models are fitted on the sessions that started and ended before it and predict it. The meta-model learns from

these predictions, the base models are refitted on the whole window, and the meta-model combines their predictions for the validation quarter. Sessions whose label is not yet known at the start of the validation quarter are removed, as in the purge of Task 2, and assertions in the code check the timing of every inner fit. The feature set and base-model configurations were selected using the development folds; these checks concern how the meta-training predictions are constructed, not an independent estimate of the full selection procedure. The coefficient on the Naive Bayes logit moves from 0.023 in fold 1 to −0.001 in fold 4, while logistic regression has coefficients from 0.56 to 0.66 and the tree from 0.31 to 0.47. These are coefficients on correlated logit scores, not percentages of a weighted average. Their pattern suggests little additional contribution from Naive Bayes once the other two predictions are known.

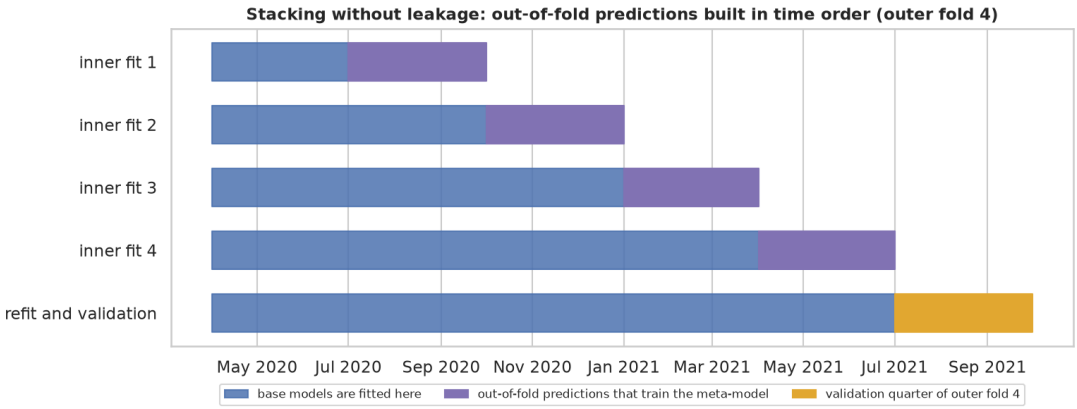


Figure 5.2: Stacking without leakage: the out-of-fold predictions that train the meta-model are built in time order inside the training window (outer fold 4)

All three ensembles improve on their constituents in every fold. The random forest is +0.025 above the tree and gradient boosting +0.028, consistent with the benefits of averaging and sequential correction. Their tree settings also differ from the single CART model, so this comparison does not isolate variance and bias separately. The stacking is above the logistic regression by +0.018, the tree by +0.013 and Naive Bayes by +0.085, but its gain over the best constituent is the smallest of the three, and in practice it mixes the linear model and the tree. Gradient boosting and the random forest are within the practical tolerance (+0.0034 for boosting, ahead in three folds).

5.4 Comparison and alerts

Table 5.3 ranks all models with the rules of Section 4.6, and Figure 5.3 shows them fold by fold.

Gradient boosting (0.626) and the random forest (0.622) lead the ranking and differ by less than our practical tolerance. We prefer gradient boosting, because it is ahead in three folds and much cheaper to refit when the failure pattern changes. It beats B1 in every fold, by +0.103 to +0.120 (+0.115 on average), with a lift of 3.50 against 2.85. The standard deviation across folds is about the same for every model, B1 included, and fold 2 is the worst for all of them: the quarters differ in difficulty for all models alike. This shared pattern does not show whether prevalence, predictor changes or training-window size explains the improvement over time. The weighted means are only 0.004 to 0.007 above the unweighted ones, so the largest fold does not decide the ranking.

Table 5.3: All models and baselines on the four validation quarters

Model	PR-AUC Mean	PR-AUC Weighted	PR-AUC Std	PR-AUC Worst fold	ROC-AUC	Brier	Folds won vs B1
Gradient boosting	0.626	0.632	0.038	0.575	0.836	0.102	4 of 4
Random forest	0.622	0.629	0.042	0.567	0.835	0.102	4 of 4
Stacking	0.610	0.617	0.041	0.556	0.831	0.104	4 of 4
Decision tree	0.597	0.604	0.041	0.544	0.824	0.105	4 of 4
Logistic regression	0.593	0.599	0.042	0.534	0.825	0.106	4 of 4
Naive Bayes	0.525	0.531	0.037	0.472	0.814	0.172	3 of 4
B1 post failure rate	0.511	0.516	0.042	0.456	0.772	0.118	–
B0 no skill	0.181	0.185	0.021	0.162	0.504	0.148	–

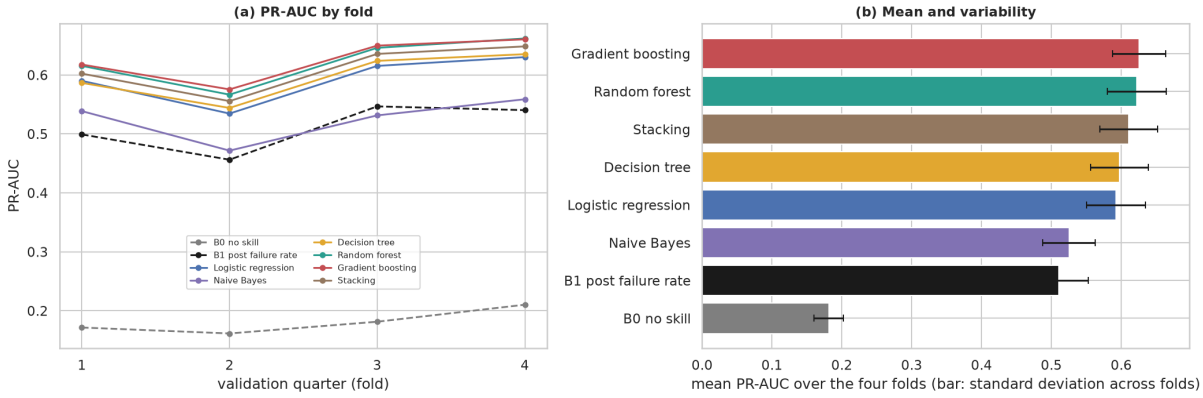


Figure 5.3: All models and baselines: (a) PR-AUC by fold; (b) mean PR-AUC with the standard deviation across folds

Table 5.4: Operational results of gradient boosting and B1 (mean of the four folds; per 1,000 sessions)

Budget	Model	Precision	Recall	Correct alerts	False alerts	Missed failures
5%	Gradient boosting	0.880	0.246	44.0	6.0	136.1
	B1	0.738	0.207	36.9	13.1	143.2
10%	Gradient boosting	0.724	0.404	72.4	27.6	107.7
	B1	0.594	0.330	59.4	40.6	120.7
20%	Gradient boosting	0.528	0.586	105.5	94.5	74.5
	B1	0.451	0.499	90.2	109.8	89.9

Table 5.4 shows what this means for an operator. With alerts on 5% of the sessions, 88% of the alerts of gradient boosting are real failures, against 74% for B1. Per 1,000 sessions, it gives 7 fewer false alerts and catches 7 more failures. The gain is mostly in precision at small budgets (+14 points at 5%, +8 at 20%). Even so, the model misses most failures when only 5% of the sessions can be reviewed (136 of about 180 per 1,000). The dominance of post history suggests that failures without a recent history may be harder to identify, but this comparison does not directly test that explanation.

6 Optimisation, Threshold and Final Evaluation

In the fourth task we tuned the most promising models, chose the final model and its cut-off, and only then scored the test period [1]. Every choice up to Section 6.5 uses the four development folds of Chapter 4. The model, the 27 features and the cut-off were written to file before the test rows were read, and the test period was scored once, with nothing changed afterwards.

6.1 Models chosen and search

We optimised gradient boosting, the random forest and the decision tree. They are the two best ensembles and the best individual model of Table 5.3, and they cover three principles: boosting, bagging with random feature subsets, and a single tree. Gradient boosting and the forest were only 0.003 apart, inside our tolerance, so both needed a search before we named the final model. The tree's two complexity controls, depth and minimum leaf size, are the settings the predictive-models slides say to choose on validation data [14]. We left out stacking, which is 0.015 behind and would need three base models and a meta-model tuned in the inner loop; logistic regression, whose two penalties already gave the same PR-AUC in every fold; and Naive Bayes, whose only setting does not touch the independence assumption behind its gap.

Each model was searched on an explicit grid (Table 6.1). Every configuration was fitted on the training window of each fold and scored on the quarter that follows, so the search respects time order and never sees the test period. The objective is the unweighted mean PR-AUC of the four folds. We used the selection rule of Task 3: among the configurations within 0.005 of the best mean, keep the simplest. When the choice sat on the edge of a range, we widened that range once. Grid search is not in the slides; we used it because it states the search space and the procedure exactly, which the brief asks us to report, and with at most 41 configurations per model a random search would not cover the space better [6].

Table 6.1: Search spaces and selected configurations (values marked * were added by the edge rule; 39, 13 and 41 configurations were tried, 36 minutes in total)

Model	Hyperparameter	Values tried	Search choice	Task 3
Gradient boosting	maximum depth	3*, 4, 6, 8	4	6
	learning rate	0.05, 0.1	0.1	0.1
	iterations	100*, 200, 400	100	200
	minimum sessions per leaf	50, 100, 200, 400*	200	100
Random forest	minimum sessions per leaf	10, 20, 50, 100, 200*	100	50
	features per split	5 ($\sqrt{27}$), 9, 13	9	5
Decision tree	maximum depth	6, 8, 10, 12, 16	8	8
	minimum sessions per leaf	50, 100, 200, 400, 800*	400	100
	split criterion	Gini, entropy	entropy	Gini

For boosting and the forest the results are flat. Of the 39 boosting configurations, 31

lie within 0.005 of the best (0.627), and 11 of the 13 forest configurations lie within 0.0031 of theirs. A smaller learning rate or more features per split help slightly, but by less than the tolerance. The tree is the only model where the grid matters. Depth 6 underfits (about 0.585), and from depth 8 on the leaf size decides: with leaves of 400 sessions, depths 10 to 16 reach 0.603 to 0.604, while with leaves of 50 a depth-16 tree falls to 0.582 and its worst fold to 0.521, the overfitting that the minimum leaf size is meant to prevent [14]. The boosting choice stayed on the smallest number of iterations after one widening; every configuration near it is inside the tolerance.

6.2 Optimised against initial, and the final model

We adopted a search choice only if it beat the Task 3 configuration by more than 0.005 on average (meaningful) and in at least 3 of the 4 folds (stable). Table 6.2 shows that no model passes. The boosting choice is simpler and slightly worse, winning one fold. The forest choice wins all four folds, so it is stable, but its gain of +0.001 is far below the tolerance. The tree gains +0.002 in three folds, but its worst fold drops by 0.003. The best tree of the grid is 0.006 above the Task 3 tree, but it needs depth 16 and is still 0.022 below gradient boosting. The search therefore confirms the Task 3 configurations, which already sat on the plateau.

Table 6.2: Optimised against initial configurations (mean PR-AUC over the four folds; final model in bold)

Model	Task 3	Search choice	Gain	Folds won	Best in grid	Kept
Gradient boosting	**0.626**	0.623	−0.002	1 of 4	0.627	Task 3
Random forest	0.622	0.623	+0.001	4 of 4	0.626	Task 3
Decision tree	0.597	0.599	+0.002	3 of 4	0.604	Task 3

The final model is gradient boosting with its Task 3 configuration: depth 6, learning rate 0.1, 200 iterations and at least 100 sessions per leaf [16]. Its mean PR-AUC of 0.626 is 0.003 above the random forest, inside the tolerance, so we applied the tie-break set before the search: the better worst fold (0.575 against 0.567). Boosting is also about four times faster to fit (3.1 against 12.0 seconds per fold).

6.3 Decision threshold

A score becomes an alert only once a cut-off is fixed. Of the criteria in the brief, we used the operational alert-rate constraint of the protocol: the cut-off flags 10% of the sessions, so an operator inspects about 100 sessions in every 1,000. We have no data on the cost of a missed failure against an unnecessary inspection, so a cost function would need an invented ratio, and the protocol does not select with F1 because it weights precision and recall equally without an operational reason. A minimum recall would leave the workload open. An alert budget is the one quantity an operator can set directly, and 10% is the middle of the three budgets of the protocol [8].

The cut-off was computed on the pooled out-of-fold predictions: each of the 235,472 validation sessions was scored by the final configuration fitted on its own fold's training window. The 90th percentile of these scores is 0.490. Applied to each fold, it flags 7.7% to

12.2% of the sessions, following the abnormal share of the quarter, while precision stays between 0.713 and 0.751. A fixed cut-off keeps the quality of an alert stable, and the number of alerts rises and falls with the failure level.

Table 6.3: The default cut-off of 0.50 and the selected cut-off of 0.490 on the pooled development predictions and on the test period (counts per 1,000 sessions)

	Development (out-of-fold)		Test period	
Measure	0.50	0.490	0.50	0.490
Alert rate	9.7%	10.0%	7.1%	7.4%
Precision	0.747	0.740	0.686	0.677
Recall	0.395	0.403	0.278	0.286
F1	0.517	0.522	0.395	0.402
Correct alerts	72.7	74.0	48.8	50.3
False alerts	24.6	26.0	22.4	24.0
Missed failures	111.1	109.8	127.1	125.6

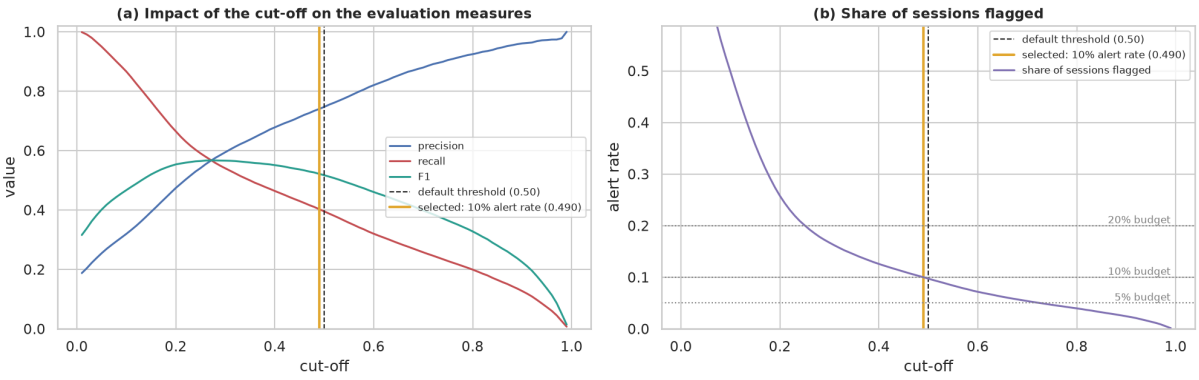


Figure 6.1: Effect of the cut-off on the pooled out-of-fold predictions: (a) precision, recall and F1; (b) share of sessions flagged, with the three alert budgets

Table 6.3 compares the two cut-offs. On development they are almost the same: the selected cut-off adds 1.3 correct and 1.4 false alerts per 1,000 sessions. This is not a property of 0.50. It happens because the scores of this model are close to probabilities (Section 6.4) and about 10% of the sessions have a predicted risk above one half. Figure 6.1 shows that F1 would peak near a cut-off of 0.3, where the model flags well over 10% of the sessions. Our cut-off comes from a stated capacity, so it would stay meaningful for a model whose scores are not calibrated.

For an operator, the 10% budget means 74.0 real failures and 26.0 false alerts in every 100 inspections per 1,000 sessions, while 109.8 of the 183.8 abnormal sessions per 1,000 are still missed. The protocol's other budgets show the trade-off: going from 5% to 10% adds 29.3 correct alerts for 20.7 false ones, while going from 10% to 20% adds 34.4 correct alerts for 65.6 false ones. Of the three budgets, 10% is the last one where the extra alerts are more often right than wrong. Read as a probability, the cut-off of 0.490 treats a missed failure as costing about the same as an unnecessary inspection (a cost ratio of 1.04), less than Section 4.3 argued; a larger budget would follow if the operator has the capacity.

6.4 Calibration

The cut-off uses the scores only as a ranking, but an operator may read them as risks, so we checked whether they behave as probabilities. Trees and boosted trees can give scores that are not probabilities [4, 11]. On the pooled out-of-fold predictions, the Brier score [10] is 0.103, against 0.150 for a constant prediction of the training prior. In ten bands of equal size (Figure 6.2), the mean predicted score and the observed abnormal share differ by at most 0.015. The model slightly under-predicts in the middle bands and is accurate in the top band, where the alerts are (0.738 predicted, 0.740 observed).

We did not add a calibration step. A calibration that keeps the order of the scores flags exactly the same 10% of the sessions, and the largest gap, under two percentage points, is small enough to read the scores as approximate risks.

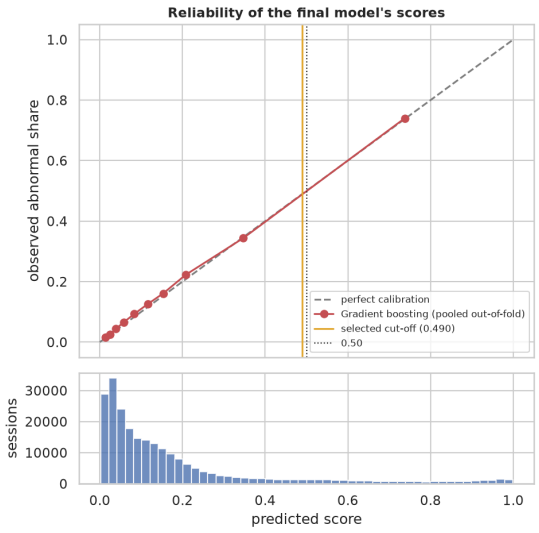


Figure 6.2: Reliability of the pooled out-of-fold scores (ten bands of equal size) and their distribution

6.5 Final evaluation on the test period

After the freeze, the final configuration was refitted on all 346,931 development sessions whose label was known before 1 October 2021; 48 sessions that ended after that date were purged, as in the folds. The test period has 80,081 sessions and an abnormal share of 17.6%. Table 6.4 gives the single evaluation, with B0 and B1 scored on the same sessions.

Table 6.4: Final evaluation on the test period (October to December 2021), with the development means for comparison (SE: bootstrap standard error of the test PR-AUC)

	Test period					Development	
Model	PR-AUC	SE	Lift	ROC-AUC	Brier	PR-AUC	ROC-AUC
Gradient boosting	0.532	0.005	3.03	0.794	0.113	0.626	0.836
B1 post failure rate	0.381	0.004	2.17	0.723	0.129	0.511	0.772
B0 no skill	0.178	0.002	1.01	0.505	0.145	0.181	0.504

The model keeps its advantage. Its test PR-AUC of 0.532 is three times the abnormal share and 0.152 above B1, a larger margin than on development (0.115). In absolute terms, however, it is 0.093 below the development mean and 0.128 below fold 4, the most recent quarter (0.660). That gap is about 27 bootstrap standard errors, so it is a real change in the last quarter of 2021 and not sampling noise, and it breaks the upward trend of the development folds. B1 falls further, from 0.540 in fold 4 to 0.381, which suggests that recent post failures predicted the next failure less well in this quarter. The ROC-AUC and the Brier score point the same way.

At the unchanged cut-off the model flags 7.4% of the test sessions instead of 10% (Table 6.3). Its alerts are less precise (0.677) and catch fewer failures (recall 0.286): per

1,000 sessions, 50.3 correct alerts, 24.0 false alerts and 125.6 missed failures. A fixed cut-off was expected to move the alert rate with the abnormal share, but 7.4% is lower than in fold 3, which had a similar share (18.2%) and 10.4% flagged, so fewer test sessions receive a high score. We report these numbers as observed and changed nothing after them.

6.6 What drives the final model

We measured how much the final model uses each group of information with permutation importance: on each validation quarter, the columns of one group are shuffled together across the sessions and the drop in PR-AUC is recorded, five times per group, with the configuration fitted on that fold's training window [15, 6]. Permutation importance is not in the slides; we used it because it works for any model and is measured on sessions the model did not see. Shuffling a group at once keeps its correlated history windows from standing in for each other.

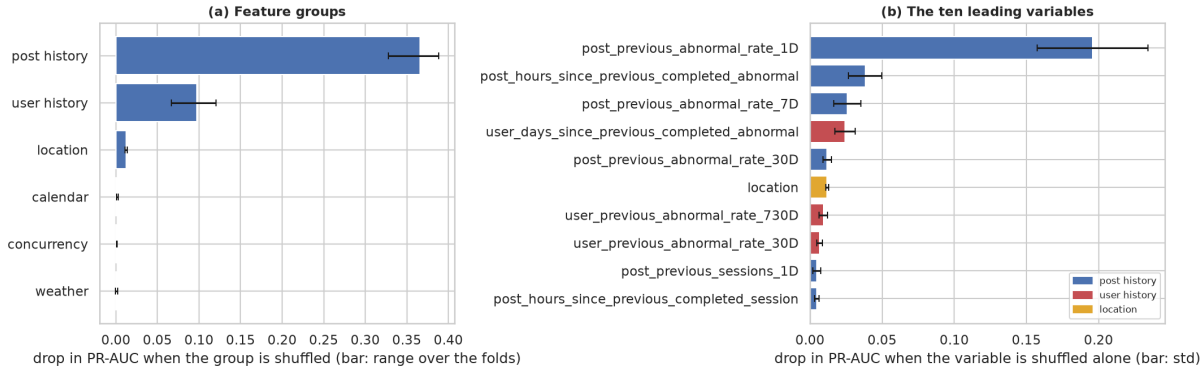


Figure 6.3: Permutation importance of the final model on the four validation quarters: (a) feature groups (bar: range over the folds); (b) the ten leading variables (bar: standard deviation)

The post history dominates (Figure 6.3). Shuffling its eight variables lowers the PR-AUC by 0.366 on average. User history comes second (0.097), and its role shrinks over time, from 0.108 in fold 1 to 0.067 in fold 4. Location costs 0.012; calendar, concurrency and weather less than 0.002 each, so the model barely uses them. The leading single variable is the post's abnormal rate over the last day (0.196), ahead of the time since the post's last abnormal session (0.038) and the 7-day rate that B1 uses (0.026).

The ranking agrees with the ablation of Section 5.1, but the sizes differ: removing post history and retraining cost 0.294, and removing user history 0.027, against permutation drops of 0.366 and 0.097. The two methods answer different questions. Permutation measures how much this fitted model uses a group; the ablation measures what is lost when the model is retrained without it, and a retrained model can partly replace user history with post history. The method has three limits here. Correlated variables share importance, so single variables are understated when the other windows are left intact. It measures use, not cause. And it describes the model on the validation quarters, before the change seen on the test period.

7 Final Model

This chapter brings the final choices together; the evidence for each stays in the chapter where it was made. Its numbers come from the last section of `Task4.ipynb`, which reads the saved results and fits nothing.

7.1 Candidate, baseline and final models

Table 7.1 lists every model we evaluated on the four validation quarters, with the baselines of Section 4.7. Only three rows have a test score, because the test period was used once.

Table 7.1: Candidate, baseline and final models: PR-AUC on the four validation quarters and on the test period. Blocks: baselines (Task 2), candidates (Task 3), tuned configurations not kept (Task 4), final model in bold. lr: learning rate; it.: iterations

		Development (four folds)				Test
Model	Configuration	Mean	Worst	vs B1	Won	PR-AUC
B0 no skill	random score	0.181	0.162	−0.329	0 of 4	0.178
B1 post failure rate	7-day rate of the post	0.511	0.456	–	–	0.381
Logistic regression	$C = 0.01$	0.593	0.534	+0.082	4 of 4	–[a]
Naive Bayes	categorical, 10 bins	0.525	0.472	+0.015	3 of 4	–[a]
Decision tree	depth 8, leaf 100	0.597	0.544	+0.087	4 of 4	–[a]
Random forest	leaf 50, $\sqrt{27}$ features	0.622	0.567	+0.112	4 of 4	–[a]
Gradient boosting	depth 6, lr 0.1, 200 it., leaf 100	0.626	0.575	+0.115	4 of 4	–[a]
Stacking	LR + NB + DT	0.610	0.556	+0.100	4 of 4	–[a]
Gradient boosting	depth 4, lr 0.1, 100 it., leaf 200	0.623	0.570	+0.113	4 of 4	–[a]
Random forest	leaf 100, 9 features	0.623	0.567	+0.113	4 of 4	–[a]
Decision tree	depth 8, leaf 400, entropy	0.599	0.541	+0.089	4 of 4	–[a]
Final model	**boosting as in Task 3**	**0.626**	**0.575**	**+0.115**	**4 of 4**	**0.532**

[a] Not scored: the test period was used once, for the final model and the two baselines.

Gradient boosting leads the candidates, 0.115 above B1 on average and better in every fold. Its lead over the random forest is 0.003, inside our tolerance of 0.005, so the choice between them rested on the worst fold, 0.575 against 0.567 (Section 6.2). None of the three tuned configurations beat its Task 3 version by more than the tolerance, so the final model is the Task 3 gradient boosting configuration. On the test period it keeps a margin of 0.152 over B1 (0.532 against 0.381), although both fall below their development level (Section 6.5).

7.2 Final preprocessing, features, hyperparameters and threshold

Table 7.2 lists what is needed to rebuild the final model. Every value equals the choices written to file on 27 October 2026, before the test period was read. The last column gives the section with the evidence.

Table 7.2: The final model

Item	Final choice	Section
Data and split	Development from 1 April 2020 to 30 September 2021, in four expanding-window folds with a purge gap; final fit on the 346,931 development sessions that ended before 1 October 2021; test from October to December 2021, scored once	3.1, 4.5
Target	Abnormal when the session ends with one of the 13 fault or failure causes; predicted when the session starts	1.1
Preprocessing	Ordinal codes for location and district, learned on the training window (an unseen category counts as missing); numeric features unchanged; no imputation, resampling or class weights	3.3
Features (27)	Post history 8, user history 10, weather 4, calendar 2, location 2, concurrency 1	5.1
Hyperparameters	Gradient boosting with depth 6, learning rate 0.1, 200 iterations and at least 100 sessions per leaf; no early stopping; seed 42	6.1, 6.2
Threshold	Cut-off 0.490, which flags 10% of the pooled out-of-fold development predictions (precision 0.740, recall 0.403)	6.3
Calibration	None; the alert budget uses the scores only as a ranking (Brier score 0.103)	6.4

7.3 Reproducibility statement

Every result in this report can be regenerated from the project repository and the course data file. The notebooks run in Python 3.12.3 with the package versions pinned in `requirements.txt`, in a fresh virtual environment. The data file, `Charging_Data_educational.csv` (about 179 MB), comes from the course channel, is not stored in the repository, and has the SHA-256 fingerprint

`f6109dc066162bc770417c39294cdecc7519646c3cfa8c1d95671ea30f0e0a9b`

Every random step uses the seed 42. The four notebooks run in the order of Table 7.3, each with a full restart and run, because each one reads the files the previous ones saved.

Table 7.3: Run order, time and files written (laptop, Intel Core Ultra 7 165H, 22 logical cores)

Notebook	Time	Content and main files written
`Task1.ipynb`	1 min	audit and preparation: feature groups, ledger, prepared data
`Task2.ipynb`	12 s	evaluation setting: protocol and baselines
`Task3.ipynb`	6 min	ablation and models: Task 3 results and fold metrics
`Task4.ipynb`	40 min	search, threshold, test: frozen choices, results, Tables 7.1–7.2

On 28 October 2026 we ran the four notebooks in a fresh clone of the repository with only the data file added. Tasks 1, 2 and 4 gave the same saved results, every test number included; Task 3 moved by less than 0.0002 in PR-AUC, which changed one value (a ROC-AUC in Table 5.1, now 0.833 instead of 0.832). The choices were frozen on 27 October 2026 at 11:40 and the test period was scored once that day. The reproduction repeated it unchanged; its notebook prints the re-run's freeze time, and the submitted file keeps the original. No model, feature, hyperparameter or threshold was changed after the test result was seen.

8 Synthesis

The project asks to what extent the information available when a charging session starts can predict that it ends abnormally. The file has no target column, so we built it from the end cause, and the dataset's own history columns reproduce our definition almost exactly (Table 1.1). The columns that separate the classes best (energy, cost and duration, with an AUC of 0.82–0.85) describe the result of the session, so they cannot be predictors. The best variable known at the start reaches 0.77. It is the recent failure history of the post: the abnormal share goes from about 4% in the lowest decile of the 7-day rate to about 60% in the highest. User history comes second (0.69–0.70), and calendar and weather add little. The records needed no repair. We found no duplicates or invalid values, the missing values only mean that there was no earlier session or failure, and the extreme values are real situations, so no row was removed or imputed. The data change over the two years: the abnormal share moves between 13.7% and 26.3%, and connection faults, which added 7.1 points in the first quarter of 2020, add only 1–2 points from the third quarter on. For this reason the sessions are split by time, the first three months are left out as a burn-in, and every score is read against the abnormal share of its own period.

Under this protocol, the 7-day failure rate of the post, used alone and without fitting anything, reaches a mean PR-AUC of 0.511 on the four validation quarters, against 0.181 for no skill. In the models of Chapter 5, recent post history provides the largest measured contribution: it lifts the PR-AUC from 0.290 with context, weather and tariff to 0.584. User history adds 0.031, with a much larger benefit for returning users and mixed changes for new users. The tariff adds no measurable improvement beyond the hour. The best model, gradient boosting on 27 features, reaches 0.626, above B1 in all four folds by 0.115 on average, and a logistic regression already reaches 0.593, suggesting that the available information limits what these models can achieve. With alerts on 5% of the sessions, 88% of its alerts are real failures, against 74% for B1, but it still catches only a quarter of the failures. Tuning did not improve on these settings (Chapter 6). On the untouched test quarter it reaches a PR-AUC of 0.532, against 0.381 for B1 and 0.178 for no skill (Table 7.1). With the cut-off fixed beforehand at a 10% development alert rate, it flags 7.4% of the test sessions: 50 correct alerts, 24 false alerts and 126 missed failures per 1,000. The margin over B1 holds, but both fall well below their development level (by 0.093 and 0.130). So the information available at the start supports a useful ranking of risky sessions, with post history providing most of the measured improvement, but the strength of that signal changes over time. Performance on failures without recent post failures is still untested.

Weather is assumed to be a start-time forecast despite its daily resolution. Durations stop at 24 h, and the order and payment timestamps remain unexplained. Every validation post was seen in training, so new-post performance is untested. User history raises privacy concerns [13]; it should only support internal session rankings, with the user-free model (0.599) available as an alternative. Feature sets, hyperparameters and the cut-off were chosen on the reported folds, so the development scores are optimistic, the ablation covers one model family, and the test period is a single quarter. The 0.005 tolerance is a practical rule; session resampling ignores dependence, and four folds are insufficient for reliable significance claims. The scores are close to calibrated, but error costs are unknown, so the 10% budget is a choice, not a measured capacity. B1's 7-day window came from Task 1's ranking, whose training period overlaps folds 1 to 3; we expect a small effect because B1 fits nothing. Finally, combining 13 fault causes can hide failure types that the model never

detects.

Bibliography

[1] Departamento de Engenharia Informática, ISEP (2026). *Mineração de Dados, First Project: Classification of Abnormal Electric-Vehicle Charging Sessions* (Tasks 1 to 4). Assignment statement, 2026/2027, 1st semester.

[2] Zhang, Y., Xu, T., Chen, T., et al. (2025). A high-resolution electric vehicle charging transaction dataset with multidimensional features in China. *Scientific Data*, 12, 643. `https://doi.org/10.1038/s41597-025-04982-1`

[3] Rodrigues, F. (2026). *Data Preparation*. Lecture slides, Mineração de Dados, DEI/ISEP, 2026/2027.

[4] Rodrigues, F. (2026). *Evaluation of Machine Learning Models*. Lecture slides, Mineração de Dados, DEI/ISEP, 2026/2027.

[5] Hyndman, R. J., Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd ed., Section 5.10, Time series cross-validation. OTexts, Melbourne. `https://otexts.com/fpp3/tscv.html`

[6] scikit-learn developers (2026). *scikit-learn User Guide*, version 1.9.1. `https://scikit-learn.org/1.9/user_guide.html`

[7] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[8] Mineração de Dados teaching team (2026). *Sampling and evaluation* (`sampling_evaluation.ipynb`). Laboratory notebook, DEI/ISEP, 2026/2027.

[9] Saito, T., Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432. `https://doi.org/10.1371/journal.pone.0118432`

[10] Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.

[11] Niculescu-Mizil, A., Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 625–632.

[12] Rodrigues, F. (2026). *Introduction to Data Science*. Lecture slides, Mineração de Dados, DEI/ISEP, 2026/2027.

[13] European Parliament and Council of the European Union (2016). Regulation (EU) 2016/679 of 27 April 2016 (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 4.5.2016, 1–88. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`

[14] Rodrigues, F. (2026). *Foundational Models for Supervised Classification*. Lecture slides, Mineração de Dados, DEI/ISEP, 2026/2027.

[15] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. `https://doi.org/10.1023/A:1010933404324`

[16] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

[17] Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1